Java八股文

说明

以下为Java八股文核心内容，按MD标准排版，复制全部文本粘贴到 `.md` 文件即可使用（兼容Typora、Obsidian、VS Code等所有MD编辑器，可直接用于背诵、笔记整理）。

一、Java基础核心（必背）

1.1 Java语言的特点

1. 跨平台性：依托JVM，一次编译（.class文件），多处运行（Windows、Linux等），JVM屏蔽底层系统差异；
2. 面向对象：封装、继承、多态三大核心特性，万物皆对象（除基本数据类型，对应包装类）；
3. 安全性：无指针操作、自动垃圾回收（GC）、异常处理机制，避免内存泄漏和非法访问；
4. 健壮性：强类型语言，编译期检查语法错误，运行期异常可捕获，减少程序崩溃风险；
5. 可移植性、多线程、动态性（反射）等辅助特性。

1.2 基本数据类型与包装类

基本数据类型（8种，按占用字节排序）

| 类型 | 占用字节 | 取值范围 | 包装类 |

| :--- | :--- | :--- | :--- |

| byte | 1 | -128 ~ 127 | Byte |

| short | 2 | -32768 ~ 32767 | Short |

| int | 4 | -2^{31} ~ $2^{31}-1$ | Integer |

| long | 8 | -2^{63} ~ $2^{63}-1$ | Long |

| float | 4 | 单精度浮点数（精度6-7位） | Float |

| double | 8 | 双精度浮点数（精度15-17位） | Double |

| char | 2 | 0 ~ 65535（Unicode编码） | Character |

| boolean | 1（JVM中实际占用1字节） | true / false | Boolean |

核心考点

- 自动装箱/拆箱：基本类型 ↔ 包装类的自动转换（如 int → Integer），底层依赖 `valueOf()` 方法；
- 缓存机制：Integer、Byte等包装类，对[-128, 127]范围内的值进行缓存，超出范围则new新对象；
- 注意：boolean不能参与算术运算，char本质是Unicode编码，可直接赋值数字（如 char c = 65 → 'A'）。

1.3 == 与 equals() 的区别

1. == ：

- 基本数据类型：比较的是**值**是否相等；
- 引用数据类型：比较的是**内存地址**是否相等（是否为同一个对象）。

2. equals()：

- 是Object类的方法，默认实现是 `return this == obj;`（即比较地址）；
- 常用类（String、Integer等）重写了equals()，改为比较**内容**是否相等；
- 自定义类重写equals()时，需遵循：自反性、对称性、传递性、一致性，同时建议重写hashCode()（保证equals相等的对象，hashCode一定相等）。

1.4 String、StringBuffer、StringBuilder 的区别

| 特性 | String | StringBuffer | StringBuilder |

| :--- | :--- | :--- | :--- |

| 可变性 | 不可变（底层是final char数组） | 可变 | 可变 |

| 线程安全 | 安全（不可变本身线程安全） | 安全（方法加synchronized） | 不安全 |

| 效率 | 最低（拼接时创建新对象） | 中等 | 最高 |

| 适用场景 | 字符串不频繁修改（如常量、少量拼接） | 多线程环境下字符串修改 | 单线程环境下字符串频繁修改 |

> 补充：String拼接底层，JDK1.8后会自动优化为StringBuilder，但循环拼接仍建议手动使用StringBuilder（避免多次创建对象）。

1.5 异常体系

异常层级结构

Object → Throwable（顶层）

├─ Error（错误，JVM层面，无法捕获处理，如OutOfMemoryError、StackOverflowError）

└─ Exception（异常，可捕获处理）

├─ checked异常（编译期异常，必须处理，如IOException、SQLException）

└─ unchecked异常（运行时异常，无需强制处理，如NullPointerException、ArrayIndexOutOfBoundsException）

核心考点

- 异常处理方式：try-catch-finally、throws（声明异常，交给调用者处理）；
- finally特性：无论try/catch是否执行，finally都会执行（除非JVM退出，如System.exit(0)）；
- try-with-resources：JDK7+新增，自动关闭实现AutoCloseable接口的资源（如流、连接），无需手动finally关闭；
- throw vs throws：throw是手动抛出异常对象，throws是声明方法可能抛出的异常。

二、面向对象核心（高频）

2.1 封装、继承、多态

1. 封装

- 定义：隐藏对象的属性和实现细节，仅对外提供公共访问方式（getter/setter）；
- 目的：提高代码安全性、可维护性，避免外部直接修改属性；
- 实现：用private修饰属性，提供public的getter/setter方法。

2. 继承

- 定义：子类继承父类的属性和方法，同时可扩展自身的属性和方法；
- 关键字：extends（单继承，Java不支持多继承，可通过接口实现多继承效果）；
- 核心：super关键字（调用父类构造器、父类方法），子类构造器默认调用父类无参构造器（若父类无无参构造，子类必须显式调用父类有参构造）；
- 注意：final类不能被继承，final方法不能被重写。

3. 多态

- 定义：同一行为，不同对象有不同的实现方式（父类引用指向子类对象）；
- 实现条件：① 子类继承父类；② 子类重写父类方法；③ 父类引用指向子类对象；
- 核心：编译期看父类，运行期看子类（动态绑定）；
- 补充：多态无法调用子类独有的方法，需强制转换为子类类型（向下转型），转型前建议用instanceof判断。

2.2 抽象类与接口的区别

| 特性 | 抽象类（abstract class） | 接口（interface） |

| :--- | :--- | :--- |

| 继承方式 | 子类用extends继承，单继承 | 类用implements实现，多实现 |

| 方法修饰 | 可包含抽象方法（abstract）、普通方法、静态方法 | JDK8前：仅抽象方法；JDK8+：可含默认方法（default）、静态方法 |

| 属性修饰 | 可含各种修饰符（private、public等）的属性 | 仅public static final（常量），默认省略修饰符 |

| 构造器 | 有构造器（供子类继承调用） | 无构造器 |

| 设计目的 | 抽取子类共性，定义类的模板 | 定义行为规范，实现多继承，解耦 |

2.3 静态关键字（static）

1. 可修饰：属性、方法、代码块、内部类；
2. 核心特性：属于类，不属于对象，优先于对象加载（类加载时初始化）；
3. 注意事项：

- 静态方法不能调用非静态属性/方法（非静态依赖对象，静态无对象）；
- 非静态方法可以调用静态属性/方法；

- 静态代码块：类加载时执行一次，用于初始化静态属性，优先级高于构造器；
- 静态内部类：不依赖外部类对象，可直接创建实例（外部类名.内部类名 实例名 = new 外部类名.内部类名()）。

2.4 重写（Override）与重载（Overload）的区别

| 特性 | 重写（Override） | 重载（Overload） |

| :--- | :--- | :--- |

| 定义 | 子类重写父类的方法，方法名、参数列表、返回值完全一致 | 同一类中，方法名相同，参数列表（个数、类型、顺序）不同 |

| 范围 | 子类与父类之间 | 同一类中（或子类继承父类后，与父类方法构成重载） |

| 修饰符 | 子类方法访问权限不能低于父类（如父类public，子类不能private） | 无访问权限限制 |

| 返回值 | 子类返回值不能大于父类返回值范围（协变返回值） | 无要求，可不同 |

| 异常 | 子类方法抛出的异常不能多于/大于父类方法 | 无要求 |

三、集合框架（重中之重）

3.1 集合框架整体结构

Collection（接口，存储单值）

├─ List（有序、可重复、有索引）：ArrayList、LinkedList、Vector

├─ Set（无序、不可重复）：HashSet、LinkedHashSet、TreeSet

└─ Queue（队列，先进先出）：LinkedList、PriorityQueue

Map（接口，存储键值对）：HashMap、LinkedHashMap、TreeMap、Hashtable、ConcurrentHashMap

3.2 List接口核心实现类对比

| 类名 | 底层结构 | 线程安全 | 查询效率 | 增删效率 | 适用场景 |

| :--- | :--- | :--- | :--- | :--- | :--- |

| ArrayList | 动态数组（Object[]） | 不安全 | 高（随机访问，O(1)） | 低（需扩容、移动元素，O(n)） | 大量查询、少量增删 |

| LinkedList | 双向链表 | 不安全 | 低（遍历访问，O(n)） | 高（直接操作节点，O(1)） | 大量增删、少量查询 |

| Vector | 动态数组（Object[]） | 安全（方法加synchronized） | 高 | 低（扩容效率低于ArrayList） | 多线程、少量增删查询 |

> 补充：ArrayList扩容机制：JDK1.8中，初始容量10，扩容时按1.5倍扩容（oldCapacity + (oldCapacity >> 1)）。

3.3 Set接口核心实现类对比

| 类名 | 底层结构 | 排序 | 线程安全 | 去重原理 |

| :--- | :--- | :--- | :--- | :--- |

| HashSet | 哈希表（数组+链表/红黑树） | 无序 | 不安全 | 先比较hashCode，再比较equals（两者都相等则视为重复） |

| LinkedHashSet | 哈希表+双向链表 | 插入顺序 | 不安全 | 继承HashSet，额外用链表维护顺序 |

| TreeSet | 红黑树 | 自然排序/自定义排序 | 不安全 | 基于Comparable接口（compareTo()）或Comparator接口排序去重 |

3.4 Map接口核心实现类对比

| 类名 | 底层结构 | 线程安全 | 允许null键/值 | 适用场景 |

| :--- | :--- | :--- | :--- | :--- |

| HashMap | JDK1.8：数组+链表（长度≤8）/红黑树（长度>8） | 不安全 | 允许1个null键、多个null值 | 单线程，高频使用 |

| Hashtable | 哈希表（数组+链表） | 安全（方法加synchronized） | 不允许null键/值 | 多线程（已被ConcurrentHashMap替代） |

| LinkedHashMap | 哈希表+双向链表 | 不安全 | 允许1个null键、多个null值 | 需要按插入顺序/访问顺序遍历 |

| TreeMap | 红黑树 | 不安全 | 不允许null键，允许null值 | 需要排序的键值对场景 |

| ConcurrentHashMap | JDK1.8：数组+链表/红黑树（CAS+synchronized） | 安全 | 不允许null键/值 | 高并发场景（高效线程安全） |

HashMap核心考点（高频面试）

1. 底层结构：JDK1.7是数组+链表（拉链法），JDK1.8优化为数组+链表+红黑树（链表长度>8且数组长度≥64时，链表转红黑树；长度≤6时，红黑树转链表）；
2. 哈希冲突解决：拉链法（链表/红黑树存储冲突元素）；
3. 扩容机制：初始容量16，负载因子0.75，当元素个数（size）> 容量×负载因子时，扩容为原来的2倍；
4. 键去重：同HashSet，先比较hashCode，再比较equals。

四、多线程（高频难点）

4.1 线程的创建方式（4种）

1. 继承Thread类，重写run()方法：

```

class MyThread extends Thread {

@Override

public void run() {
```

```

System.out.println("线程执行");

}

}

// 启动：new MyThread().start();

```

2. 实现Runnable接口，重写run()方法（推荐，避免单继承限制）：

```

class MyRunnable implements Runnable {

@Override

public void run() {

System.out.println("线程执行");

}

}

// 启动：new Thread(new MyRunnable()).start();

```

3. 实现Callable接口，重写call()方法（可返回值、可抛异常）：

```

class MyCallable implements Callable<Integer> {

@Override

public Integer call() throws Exception {

return 100; // 返回值

}

}

// 启动：

FutureTask&lt;Integer&gt; task = new FutureTask<>(new MyCallable());

new Thread(task).start();

Integer result = task.get(); // 获取返回值（会阻塞）

```

4. 线程池创建（实际开发首选，控制线程数量，避免资源耗尽）：

```java

// 1. 创建固定线程池

ExecutorService pool = Executors.newFixedThreadPool(5);

// 2. 提交任务

pool.submit(new MyRunnable());

// 3. 关闭线程池

pool.shutdown();

```

4.2 线程状态（生命周期）

1. 新建（New）：线程对象创建，未调用start()；
2. 就绪（Runnable）：调用start()，线程进入就绪队列，等待CPU调度；
3. 运行（Running）：CPU调度线程，执行run()方法；
4. 阻塞（Blocked）：线程暂时停止执行（如sleep()、wait()、synchronized等待）；
5. 终止（Terminated）：run()方法执行完毕，或线程异常终止。

4.3 线程同步（解决线程安全问题）

1. synchronized关键字（悲观锁）

- 作用：保证同一时刻，只有一个线程执行同步代码块/方法，避免并发修改共享资源；
- 用法：修饰方法（非静态：锁当前对象；静态：锁类对象）、修饰代码块（锁指定对象，如synchronized (this)）；
- 底层：依赖JVM的监视器锁（monitor），通过对象头的锁标志位实现。

2. Lock锁（乐观锁，JDK1.5+）

- 核心实现：ReentrantLock（可重入锁），替代synchronized，更灵活；
- 常用方法：lock()（加锁）、unlock()（释放锁，需放在finally中）、tryLock()（尝试加锁，不阻塞）；
- 优势：可中断锁、可超时锁、公平锁/非公平锁（默认非公平），适合高并发场景。

3. 其他同步方式

- volatile关键字：保证变量可见性、禁止指令重排序，但不保证原子性（不能解决多线程修改问题）；
- 原子类：java.util.concurrent.atomic包下，如AtomicInteger，通过CAS（比较并交换）实现原子操作，无锁高效；
- 线程池：通过控制线程数量，减少线程创建销毁开销，间接保证线程安全。

4.4 线程通信（wait()、notify()、notifyAll()）

- 前提：必须在synchronized同步代码块/方法中使用（获取对象锁后才能调用）；
- wait()：释放当前对象锁，让线程进入等待队列，等待被唤醒；
- notify()：唤醒等待队列中一个随机线程，使其进入就绪状态；
- notifyAll()：唤醒等待队列中所有线程，使其进入就绪状态；

> 注意：wait()和sleep()的区别：wait()释放锁，sleep()不释放锁；wait()需被notify()唤醒，sleep()到时间自动唤醒。

五、JVM基础（核心考点）

5.1 JVM内存结构（JDK1.8）

1. 方法区（Method Area）：存储类信息、常量、静态变量、编译后的字节码等（JDK1.8用元空间Metaspace替代，元空间使用本地内存，不是JVM内存）；
2. 堆（Heap）：最大的内存区域，存储对象实例和数组，是GC（垃圾回收）的主要区域，分为年轻代（Eden区、Survivor区：From/To）和老年代；
3. 虚拟机栈（VM Stack）：每个线程对应一个栈，存储栈帧（方法调用时创建），栈帧包含局部变量表、操作数栈、返回地址等，线程结束栈销毁；
4. 本地方法栈（Native Method Stack）：存储本地方法（native修饰的方法）的调用信息，与虚拟机栈类似；
5. 程序计数器（Program Counter Register）：记录当前线程执行的字节码指令地址，线程私有，唯一不会OOM的区域。

5.2 垃圾回收（GC）

1. 垃圾判断标准

- 引用计数法：对象被引用一次，计数+1，计数为0则为垃圾（缺点：无法解决循环引用问题）；
- 可达性分析算法（JVM采用）：以GC Roots为起点，遍历对象引用链，无法到达的对象即为垃圾（GC Roots：虚拟机栈局部变量、方法区静态变量、本地方法栈本地变量等）。

2. 垃圾回收算法

- 标记-清除算法：先标记垃圾对象，再清除（缺点：产生内存碎片）；
- 复制算法：将内存分为两块，存活对象复制到另一块，清除原块（优点：无内存碎片；缺点：内存利用率低，适合年轻代）；
- 标记-整理算法：标记存活对象，将其移动到内存一端，清除剩余部分（优点：无内存碎片，适合老年代）；
- 分代回收算法（JVM实际采用）：年轻代用复制算法（存活率低），老年代用标记-整理算法（存活率高）。

3. 常用垃圾收集器

- 年轻代：SerialGC（串行GC，单线程，适合小型应用）、ParNewGC（并行GC，多线程，配合老年代CMS）；
- 老年代：CMS（并发标记清除，低延迟，适合响应时间优先）、G1GC（区域化分代回收，兼顾吞吐量和延迟，JDK1.7+）、ZGC（超低延迟，大内存场景）。

5.3 类加载机制

1. 类加载流程（3个阶段）

1. 加载：通过类的全限定名，获取类的字节码文件，将其转换为方法区的运行时数据结构，创建Class对象；
2. 链接：分为验证（验证字节码合法性）、准备（为静态变量分配内存，设置默认值）、解析（将符号引用转换为直接引用）；
3. 初始化：执行静态代码块，为静态变量赋值（执行类构造器()方法）。

2. 类加载器（双亲委派模型）

- 核心：子类加载器加载类时，先委托父类加载器加载，父类无法加载时，子类才自己加载；
- 层级（从顶层到底层）：

1. 启动类加载器（Bootstrap ClassLoader）：加载JDK核心类（如rt.jar），由C++实现；
2. 扩展类加载器（Extension ClassLoader）：加载JDK扩展类（如ext目录下的jar）；
3. 应用类加载器（Application ClassLoader）：加载项目中的类（classpath下的类）；
4. 自定义类加载器：继承ClassLoader，重写findClass()方法，实现自定义加载逻辑（如加密类加载）。

六、备注（使用说明）

1. 本文档涵盖Java八股核心考点，适合面试背诵、笔记整理，可直接复制保存为.md文件使用；
2. 代码块可直接复制到IDE中运行，示例清晰，贴合面试高频提问场景；
3. 可根据自身需求，补充Spring、MyBatis等框架八股内容，直接在对应位置添加即可；
4. 兼容所有MD编辑器，支持折叠、搜索，方便快速查找知识点。